CE-355 Lab 2
Robotic Hand Controller

1. Introduction

FPGAs can be used to control physical devices by generating precisely timed digital signals in response to external inputs. In this lab, you will build a robotic hand controller that uses switches as inputs, LEDs for visual feedback, and five servo motors as physical outputs.

The lab focuses on fundamental building blocks of FPGA-based control systems: **combinational processes, sequential statements, concurrent statements,** and **for-generate statements**. Controlling motors also requires precise timing. The FPGA's clock drives counters that measure time and generate timing intervals, and these are built with clocked processes. Clocked processes are covered in a later lab. In this lab they are provided for you in the starter code, so you can focus on the combinational logic that decides what the hand does.

You will also use **for-generate** statements to create multiple instances of the same hardware module. This provides modularity and automatic expansion of parallel hardware structures, allowing five servo controllers to be created from a single module without duplicating the VHDL code.

Starter VHDL files are provided with **TODO sections** that you will complete to implement the gesture controller and integrate the five servo controllers. You will then simulate, synthesize, program, and test the completed system on the DE25-Standard.

By completing this lab, you will:

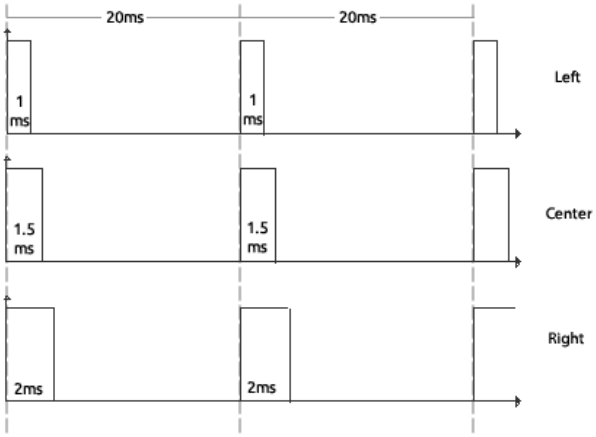
- Implement combinational control logic using processes and sequential statements.
- Use for-generate statements to create modular, parallel hardware.
- Generate PWM signals to control servo motors.
- Interface FPGA logic with switches, LEDs, and external motors.
- Simulate, program, and test a real-time FPGA control system.

2. Background: PWM and Servo Control

Pulse Width Modulation (PWM) is a technique used to control electronic devices by varying the amount of time that a digital signal remains high during a fixed period. Although the signal has only two logic levels, the duration of the high portion of the signal can be used to represent a control value.

Servo motors use PWM signals to specify their position. For the servo used in this lab, a complete PWM cycle is **20 ms**, corresponding to a frequency of **50 Hz.** The width of the high pulse determines the desired servo position. A shorter pulse moves the servo toward one end of its range, while a longer pulse moves it toward the other end. For the LFD-01 servo used in this lab, the relationship between pulse width and position is approximately:

Servo Position	Pulse Width
0°	0.5 ms
45°	1.0 ms
90°	1.5 ms
135°	2.0 ms
180°	2.5 ms



Thus, for a PWM pulse range of 0.5-2.5 ms, if the signal is set high for 0.5ms out of a total of 20ms, the servo will move completely to 0°. If the signal is set high for 2.5ms out of a total of 20ms, the servo will turn to 180°.

The FPGA operates using a **50 MHz clock**, requiring a counter from 0 to 10^6-1 to adjust the pulse interval. The following equation can be used to calculate the number of counts out of 10^6 that the signal should be high:

$$(\boldsymbol{Min\ PWM\ Pulse} + \frac{\boldsymbol{Angle}}{\boldsymbol{Rotate\ Range}} \times \boldsymbol{PWM\ Pulse\ Range}(\boldsymbol{ms})) \times \frac{\boldsymbol{10^6}}{\boldsymbol{20}(\boldsymbol{ms})}$$

Based on the Servo Specification, this servo's rotation range is 180°, and its PWM pulse range is between 0.5 and 2.5ms. Hence, the count of the signal that must be high in order to rotate the servo 45 degrees is:

$$(0.5ms + \frac{45}{180} \times (2.5ms - 0.5ms)) \times \frac{10^6}{20(ms)} = 50{,}000$$

This indicates that, for each 10^6 Count, the PWM signal should be high for 50,000 cycles and low for the remaining 950,000 cycles.

The provided `pwm_servo` module performs this timing and PWM generation. In this lab, you will use the servo position provided by the `gesture_controller` to control the PWM output for each finger.

Important: The PWM signal is a **control signal** and should not be used to power the servo. The servo requires an appropriate external 12V-5A power connection, while the FPGA GPIO provides the PWM control signal.

3. Project Setup

You will use **ModelSim** to simulate and verify your designs. You will use **Altera Quartus Prime** to compile, synthesize, and program your design on the DE25-Standard FPGA development board.

Maintaining an organized file structure is an important part of RTL design. Each directory has a specific purpose. For each lab, organize your project using the following directory structure:

```
Lab 2/
│
├── vhdl/   — Contains the VHDL source files for the design.
├── sim/    — Contains the ModelSim testbench and simulation files.
└── syn/    — Contains the Altera Quartus Prime synthesis files.
```

The provided VHDL files include TODO sections that you will complete during the lab. The main design files are:

- `pwm_servo.vhd` — Provided servo PWM controller.
- `gesture_controller.vhd` — Gesture selection and motion control.
- `robotic_hand.vhd` — Top-level design connecting the controller and five servos.
- `robotic_hand_tb.vhd` — Testbench used to simulate the complete design.

*NOTE: Throughout this assignment, please refer to the **DE25-Standard Development Kit User Manual,** the **DE25-Standard Getting Started Guide,** and the **Servo Motor Kit manual** which contains more details on the board set up, pinouts, and configuration.*

4. System Design

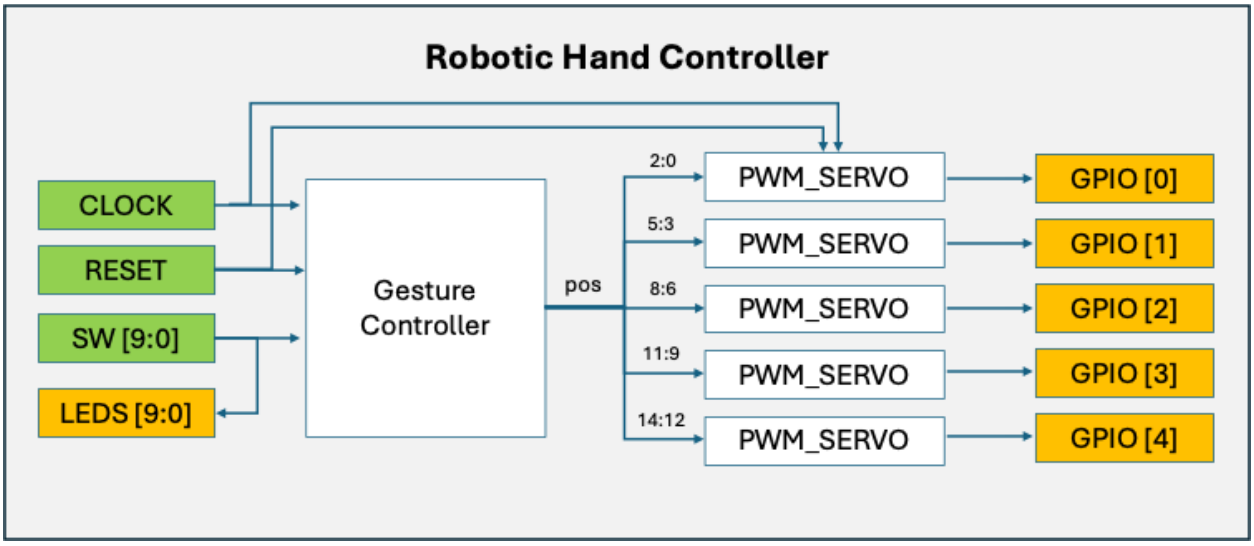
The robotic hand controller is constructed from several hardware modules that work together to control five servo motors. The system uses the FPGA clock as a common timing reference and responds to switch inputs to select the desired hand gesture.

The overall system consists of three functional parts:

- **Gesture Controller** — Reads the switch inputs, selects the appropriate hand pose, and generates the five servo position commands. It also uses counters to control gestures that require continuous movement.
- **Servo PWM Controllers** — Generates the PWM control signal for each servo based on its assigned position. Five instances of the provided `pwm_servo` module are used, one for each finger.
- **Top-Level Robotic Hand Controller** — Connects the switches, LEDs, gesture controller, and five servo controllers into a complete system.

The `gesture_controller` produces a 15-bit position vector containing five 3-bit servo position values. The top-level `robotic_hand` design uses a for-generate statement to replicate the `pwm_servo` module five times. Each instance receives one 3-bit position value from the gesture controller and produces one PWM output for a servo. The switches are also connected to the LEDs so that the selected switch inputs can be observed directly on the board. The five PWM outputs are connected to the DE25-Standard GPIO header and then to the servo motors.

The complete design therefore combines **synchronous control logic**, **timing counters**, **parallel hardware replication**, and **external device interfaces** to create a real-time robotic hand controller.



5. Design Implementation

In this section, you will complete the provided VHDL files to implement the robotic hand controller. The starter files contain `TODO` sections that identify the portions of the design you are responsible for completing.

The design consists of three VHDL entities:

- **pwm_servo** — Provided with the lab. Generates the PWM signal used to control one servo motor.
- **gesture_controller** — You will complete this module to decode the switch inputs, select servo poses, and implement animated gestures using clocked processes and counters.
- **robotic_hand** — You will complete the top-level structural design by connecting the gesture controller to five instances of `pwm_servo`.

5.1 Implement the Gesture Controller

The `gesture_controller` determines which hand gesture is selected and generates the five servo positions. Excluding closed position, there are **10 gestures** in total that you need to implement, each controlled by one of the 10 switches:

- Closed position
- 1-5 Fingers
- Half open
- OK 👍
- Rock 🤘
- Open-close animation
- Finger wave animation

Each gesture is represented by a `servo_pose_t` array containing five 3-bit servo position values. Start by adding constant definitions for the static poses (**one, two, three, four, five, half-open, OK,** and **rock**) using the existing `closed_pose` as a reference. Each element of the array corresponds to one of the five servos. The 3-bit values represent the available servo positions used by the provided `pwm_servo` module. The existing `pose_to_vector` function converts the five 3-bit positions into the 15-bit `pos` output, so no changes are required to this function.

Pos (bits)	Degree	Pulse Width
"000"	0°	pulse_min = 0.5 ms
"001"	45°	pulse_45 = 1.0 ms
"010"	90°	pulse_90 = 1.5 ms
"011"	135°	pulse_135 = 2.0 ms
"100"	180°	pulse_max = 2.5 ms

The `gesture_process` is a **combinational process**. It does not use the clock. It recomputes `active_pose` whenever its inputs (`sel` and `step`) change. Inside `gesture_process`, use an if/elsif priority encoder or case statement to check `sel(0)` through `sel(9)` and set `active_pose` to the matching pose.

- If no switch is selected, set `active_pose` to `closed_pose`.
- `sel(0)` – `sel(7)` select the static poses listed above.
- `sel(8)` should alternate between `closed_pose` and `open_pose` (e.g. the five-finger pose). Use `closed_pose` on even values of `step` and `open_pose` on odd values (hint: `step mod 2`).
- `sel(9)` should make the hand wave by stepping through a sequence of finger poses as `step` counts from 0 to 9. The sequence opens the fingers one at a time and then

closes them in reverse: 0-1-2-3-4-5-4-3-2-1, then repeats. Here 0 = closed, 1 = thumb, 2 = thumb + index, 3 = thumb + index + middle, and so on.

Every branch of your if/elsif must assign **`active_pose`.** If any path through the process leaves `active_pose` unassigned, the synthesizer infers a latch, which is a common bug in combinational processes. Also, every signal your process reads must be in its sensitivity list.

The provided `counter_process` drives the timing. You do not need to change it. On each 50MHz clock edge it increments `animation_count`. When the count reaches `ANIMATION_PERIOD-1`, it resets to 0 and `step` advances from 0 to 9, then wrapping back to 0. The `ANIMATION_PERIOD` constant sets how many clock cycles each animation step lasts. You may need to change it to get a speed that looks reasonable.

5.2 Implement the Top-Level Robotic Hand

The `robotic_hand` entity is the **top-level structural design**. Its purpose is to connect the `gesture_controller` to five instances of the provided `pwm_servo` module. No control logic or clocked processes are required in this architecture. Instead, use **component declarations, internal signals, component instantiation, port maps, and concurrent assignments** to connect the modules.

First, add component declarations for `gesture_controller` and `pwm_servo` that match their entity interfaces. Declare an internal 15-bit signal to carry the five 3-bit servo position commands from the `gesture_controller`. Instantiate the `gesture_controller` and connect `clk`, `reset`, and `sw` to the corresponding top-level inputs. Connect its 15-bit `pos` output to the internal position signal.

Next, use a **for-generate statement** to create **five** instances of `pwm_servo`. Each instance should map the appropriate 3-bit section of the 15-bit position signal and connect its PWM output to the corresponding bit of the top-level `pwm` vector. The five instances should operate in parallel, with one controller dedicated to each finger. Finally, use a **concurrent assignment** to connect `sw` directly to `led`, allowing the state of all ten switches to be observed on the corresponding LEDs.

```
pwm_servos : for i in 0 to 4 generate
begin
   servo_i : pwm_servo
      generic map( ... )
      port map( ... );
end generate pwm_servos;
```

6. Simulation

Before programming the FPGA, verify the robotic hand controller using ModelSim. The testbench generates the 50 MHz clock, applies reset, selects each of the ten gestures, and observes the resulting LED and PWM outputs.

6.1 Testbench Setup

Complete the provided `robotic_hand_tb` by adding the signals, DUT component and instantiation, clock generation, reset generation, and test stimulus.

- **Signal declarations:** Add signals corresponding to all inputs and outputs of `robotic_hand`:
    - `clk` — 50 MHz clock
    - `rst` — reset input
    - `sw` — 10-bit gesture selection
    - `pwm` — five servo PWM outputs
    - `led` — 10 LED outputs

- **DUT instantiation:** Declare a `robotic_hand` component and instantiate it in the testbench. Connect each DUT port to the corresponding testbench signal. This allows the testbench to drive the clock, reset, and switches while observing the LED and PWM outputs.

- **Clock generation:** Create a clock process that continuously alternates `clk` between `'0'` and `'1'`. A 20 ns period corresponds to a 50 MHz clock, so the process should wait for 10 ns (half-cycles) between switching the clock signal. The process should repeat indefinitely.

- **Reset generation:** Create a separate process that asserts `rst` at the beginning of the simulation and then de-asserts it. Note that internally `rst` is active high, but the actual FPGA board requires active low, so you may need to add additional logic to negate the signal. Hold reset long enough for the clocked logic to initialize before beginning the gesture tests (approximately 100ns). The reset process should wait indefinitely after its initial reset state.

6.2 Test Stimulus

The ten switches represent ten different gestures. Rather than manually writing ten separate test cases, use a `for` loop to generate the test sequence. The loop should iterate from `0` through `sw'high`. During each iteration:

- Set all switch inputs to `'0'`.
- Set only the switch corresponding to the current loop index to `'1'`.

- Wait for a suitable amount of simulation time.
- Allow the next iteration to select the next gesture.

The wait time between switch selections should be long enough to clearly observe the selected pose in the waveform. For the two animated gestures, the `ANIMATION_PERIOD` may make the transitions too slow to conveniently observe. You may temporarily reduce this constant during simulation so that the animation sequence can be verified.

6.3 Create the ModelSim Command File

Create a file named `sim.do` inside the `sim` directory. This file automates the ModelSim simulation process. It should compile the VHDL source files in the correct order, start the `robotic_hand_tb` testbench, add the important signals to the waveform viewer, and run the simulation. The files should be compiled in bottom-up hierarchical order.

The sim.do file should contain commands following this structure:

```
vlib work


vcom -2008 ../vhdl/pwm_servo.vhd
vcom -2008 ../vhdl/gesture_controller.vhd
vcom -2008 ../vhdl/robotic_hand.vhd
vcom -2008 ../vhdl/robotic_hand_tb.vhd


vsim -voptargs=+acc robotic_hand_tb


add wave -group "TESTBENCH" sim:/robotic_hand_tb/*
add wave -group "DUT" sim:/robotic_hand_tb/dut/*
add wave -group "GESTURES" sim:/robotic_hand_tb/dut/gesture_inst/*


run 4000 ms
```

The commands perform the following operations:

- vlib work creates the ModelSim working library.
- vcom compiles each VHDL source file.
- vsim starts the robotic_hand_tb simulation.
- add wave adds the testbench signals to the waveform viewer.
- run executes the testbench for the specified simulation time.

6.5 Run the Simulation

Open ModelSim and change to the `sim` directory. Run the command file by entering:

```
do sim.do
```

Alternatively, the simulation can be launched directly from a terminal:

```
vsim -do sim.do
```

After the simulation completes, examine the waveform viewer and verify the relationship between the switch selection, gesture controller, servo positions, and PWM outputs.

6.3 Verify the Waveforms

Examine the `sw`, `led`, and five `pwm` signals in ModelSim. Verify that:

- Each one-hot switch selection produces the expected LED output.
- Each static gesture produces the expected combination of servo positions.
- The five PWM outputs operate independently according to their assigned positions.
- The open/close gesture alternates between its two poses.
- The wave gesture progresses through its sequence of poses.
- Reset returns the controller to the closed position.

The waveform should clearly show the relationship between **switch selection → gesture controller → servo positions → PWM outputs**.

7. Synthesis

Once the design has been successfully simulated, implement the robotic hand controller on the DE25-Standard. In this stage, you will create the Quartus Prime project, assign the FPGA pins, compile the design, and program the FPGA.

7.1 Create the Quartus Prime Project

Create a new Quartus Prime project in the `syn` directory of your lab project. Name the project `robotic_hand` and set `robotic_hand` as the **top-level entity**. Configure the project for the Agilex 5 SoC FPGA used on the DE25-Standard Device: **A5ED013BB32AE4SCS**. Add the following VHDL source files from the `vhdl` directory:

- `pwm_servo.vhd`
- `gesture_controller.vhd`
- `robotic_hand.vhd`

The robotic hand controller uses the DE25-Standard's 50 MHz system clock. Quartus Prime must be informed of the clock frequency so that it can perform timing analysis and verify that the clocked logic and counters can operate correctly. Create a new file named **robotic_hand.sdc** in the **syn** directory and add the following constraint:

```
create_clock -name clk -period "50MHz" [get_ports clk]
```

Add the **robotic_hand.sdc** file to the Quartus Prime project. The constraint defines the clk input as a 50 MHz clock and allows Quartus Prime to analyze the timing requirements of the clocked processes in the design.

7.2 Assign FPGA Pins

The top-level ports of `robotic_hand` must be assigned to the physical FPGA pins on the DE25-Standard. Open **Assignments → Pin Planner** in Quartus Prime. Assign the following signals:

Port	FPGA Pin	Description
clk	CH128	50 MHz FPGA clock
rst_n	BM78	CPU reset
sw[0]	BM62	Slide Switch 0
sw[1]	BP62	Slide Switch 1
sw[2]	BH62	Slide Switch 2
sw[3]	BH59	Slide Switch 3
sw[4]	BM59	Slide Switch 4
sw[5]	BK59	Slide Switch 5
sw[6]	BU62	Slide Switch 6
sw[7]	CF59	Slide Switch 7
sw[8]	BU59	Slide Switch 8
sw[9]	BR59	Slide Switch 9
led[0]	CC71	LED 0
led[1]	BH78	LED 1
led[2]	CH69	LED 2
led[3]	CF69	LED 3
led[4]	CA62	LED 4
led[5]	CC62	LED 5
led[6]	CF62	LED 6
led[7]	BM69	LED 7
led[8]	CA71	LED 8
led[9]	BR62	LED 9
pwm[0]	BK31	GPIO[0]

Port	FPGA Pin	Description
pwm[1]	BE43	GPIO[1]
pwm[2]	BF29	GPIO[2]
pwm[3]	BF40	GPIO[3]
pwm[4]	BK28	GPIO[4]

7.3 Compile and Synthesize

Run the complete Quartus Prime compilation flow. The compilation should complete without errors. Review the compilation messages and verify that:

- `robotic_hand` is the top-level entity.
- All required VHDL files are included.
- All top-level ports have valid pin assignments.
- The 5 PWM outputs are connected to the intended GPIO pins.
- The ten switches and ten LEDs are assigned correctly.
- The 50 MHz clock is assigned to `D8`.
- No unexpected unconnected or undriven signals are reported.
- Timing analysis completes without critical violations.
- Pay particular attention to the clock timing reports. The design contains clocked processes and counters, so the 50 MHz clock is now an important timing constraint for the system.

8. Hardware Setup and Testing

Once the design has been successfully compiled, connect the robotic hand hardware to the DE25-Standard and verify its operation. **All connections must be made with the FPGA board powered off.**

8.1 Connect the DE25-Standard and Daughter Board

***Important**: The servo requires an appropriate **external 12V-5A power supply** provided to you. Do not assume that the FPGA I/O pin can supply the servo's operating current. Verify the daughter-board power arrangement and servo power requirements before connecting the motors. Double-check the power, ground, and signal connections before turning on the system. Incorrect wiring can damage the servo, daughter board, or FPGA board.*

Connect the DE25-Standard to the computer using a USB cable through the board's **USB-Blaster** connection. Do not power on the board yet.

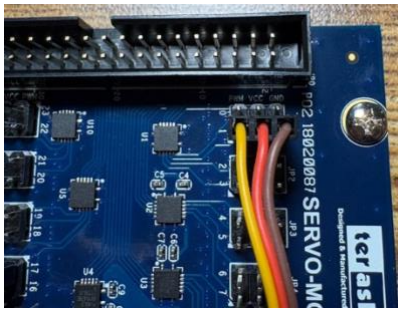
Connect the servo daughter board to the DE25-Standard's **40-pin expansion header** using the supplied ribbon cable. The IDC connectors should be firmly seated on both boards, with the

connector orientation and pin numbering aligned correctly. Verify that the corresponding pin numbers on the DE25-Standard and daughter board are aligned before applying power.

Connect the 12V power supply to the servo daughter card.

8.2 Connect the Servo Motors

Connect each servo to the corresponding output on the daughter board. A typical servo has three wires:



Servo Wire	Connection
Brown	GND
Red	VCC
Yellow	PWM signal

The five servos should be connected to the five PWM outputs assigned in the Quartus Pin Planner:

- pwm[0] → GPIO[0]
- pwm[1] → GPIO[1]
- pwm[2] → GPIO[2]
- pwm[3] → GPIO[3]
- pwm[4] → GPIO[4]

8.3 Program and Test the System

After all connections have been checked, power on the DE25-Standard. Connect the board to the computer using its USB-Blaster connection and open **Tools → Programmer** in Quartus Prime. Select the appropriate hardware connection and `.sof` file, then program the FPGA.

Once programming is complete, the FPGA will begin executing the robotic hand controller. The switches should select gestures, the LEDs should indicate the selected switch, and the five GPIO outputs should generate the PWM signals used to control the servos.

Move the switches through different configurations and verify that the corresponding hand gestures are produced. Test all ten gesture selections, including:

- One finger
- Two fingers
- Three fingers
- Four fingers
- Five fingers

- OK
- Rock 🤘
- Half-open
- Open/close animation
- Wave animation

Also test combinations in which **multiple switches are turned on simultaneously**. The `gesture_controller` uses a priority encoder, so only one gesture should be selected when multiple switches are active. Verify that the selected gesture follows the priority defined by your `if/elsif` structure, with `sw[0]` having the highest priority and `sw[9]` the lowest priority.

Finally, verify that the LEDs correctly mirror the switch positions and that each servo responds smoothly to the selected gesture. Compare the physical behavior with the ModelSim waveforms and confirm that the hardware behaves as expected.

9. Submission

Zip your folder, name it as **CE355_Lab2_Team_[N].zip** and submit your code to Canvas. Make sure to clean out any synthesis and simulation work directories.

You will also need to demo the hardware set up and simulation during TA office hours. Be prepared to explain how the system works, simulation results, and behavior of the hardware system.